

Audio Detection Agent for AI-Based Proctoring

Scope and Implementation

1. Project Overview

Project Name: Audio Detection Agent for AI-Based Proctoring

This project develops an audio analysis pipeline for an AI-based proctoring system. The system processes uploaded audio recordings, divides long recordings into manageable chunks, stores and indexes them, and uses an Audio Detection Agent to analyze each chunk and identify relevant sound events.

The agent workflow is orchestrated using **LangGraph** and follows a structured pipeline:

Retrieve → Analyze → Reason → Label → Store → Update Context → Next Chunk

2. Problem Statement

Long audio recordings cannot be efficiently analyzed as a single input. A proctoring session may contain different audio events at different timestamps, including:

- Keyboard sounds
- Human speech or talking
- Multiple or background voices
- Vehicle or traffic noise
- Environmental sounds
- Silence or no significant event

The system therefore requires a mechanism to:

1. Load audio recordings.
2. Split recordings into smaller chunks.
3. Associate metadata and timestamps with each chunk.
4. Store and index the chunks.
5. Retrieve chunks for processing.
6. Analyze and classify audio events.
7. Store structured detection results for review.

3. Project Objectives

The main objective is to build an **Audio RAG pipeline** and an **Audio Detection Agent** capable of:

- Processing uploaded audio files.

- Chunking long audio recordings.
 - Storing chunk metadata and audio information.
 - Retrieving relevant audio chunks.
 - Detecting sound events.
 - Assigning appropriate labels.
 - Maintaining context across consecutive chunks.
 - Storing structured analysis results.
 - Providing output for the larger AI-based proctoring system.
-

4. Target Users

The system is intended for:

- AI-based proctoring systems
 - Proctoring administrators
 - Monitoring teams
 - Human reviewers
-

5. System Architecture

The system consists of two primary components.

A. Audio RAG Tool

The Audio RAG Tool is responsible for managing audio data.

Its responsibilities include:

1. Loading uploaded audio files.
2. Preprocessing audio when required.
3. Splitting long recordings into chunks.
4. Generating metadata and timestamps.
5. Storing audio chunks.
6. Indexing chunks for retrieval.
7. Providing chunks to the detection agent.

B. Audio Detection Agent

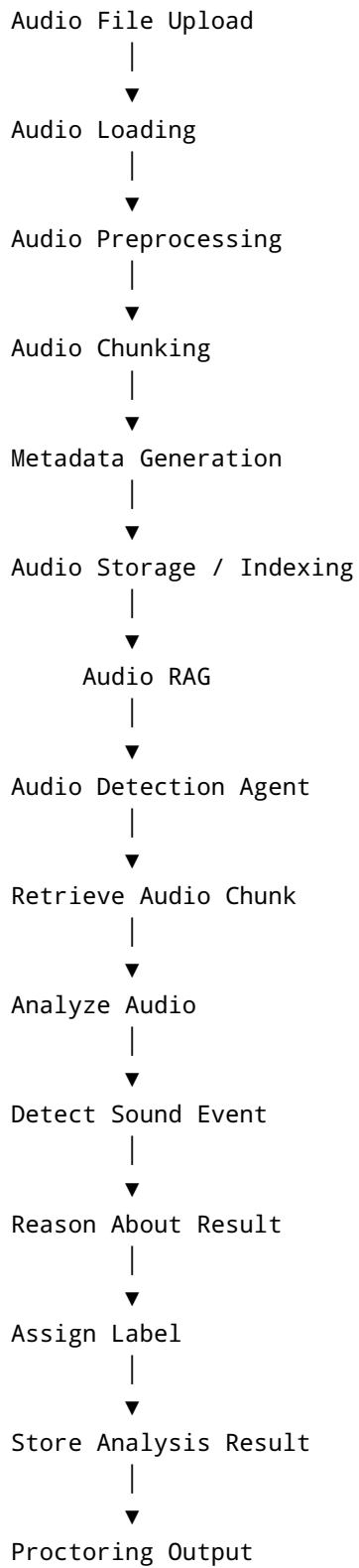
The Audio Detection Agent is responsible for processing retrieved chunks.

Its responsibilities include:

1. Retrieving an audio chunk.
2. Analyzing the audio.
3. Detecting the sound event.
4. Reasoning about the detection result.
5. Assigning a label.
6. Storing the result.

7. Updating context and processing history.
8. Moving to the next chunk when required.

Overall Architecture



6. Audio Events and Labels

The initial system will support the following event categories.

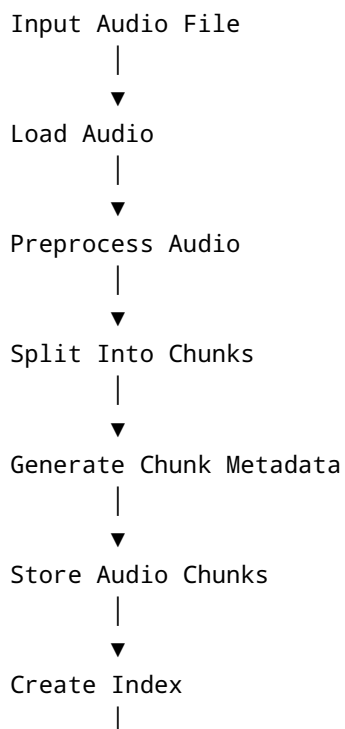
Audio Event	Label
Typing or keyboard sound	KEYBOARD
Single person speaking	HUMAN_SPEECH
Multiple or background voices	MULTIPLE_VOICES
Car, traffic, or vehicle sound	VEHICLE_NOISE
Other environmental sound	ENVIRONMENTAL_NOISE
No significant audio event	SILENCE
Unidentified sound	OTHER

The label set can be extended depending on project requirements and available training data.

7. Audio RAG Workflow

The Audio RAG component acts as the audio data source for the detection agent.

Workflow



▼
Retrieve Requested Chunk

Audio Chunk Metadata

Each chunk should contain information such as:

```
chunk_id
audio_file_id
source_file
chunk_index
start_timestamp
end_timestamp
duration
storage_path
sample_rate
processing_status
created_at
```

Example:

```
{
  "chunk_id": "chunk_00125",
  "audio_file_id": "audio_001",
  "source_file": "proctoring_session_001.wav",
  "chunk_index": 125,
  "start_timestamp": 600.0,
  "end_timestamp": 605.0,
  "duration": 5.0,
  "storage_path": "storage/audio_001/chunk_00125.wav",
  "processing_status": "PENDING"
}
```

8. Audio Chunking

Long audio recordings will be divided into fixed-duration or configurable chunks.

Example:

```
Original Audio: 60 minutes

Chunk Duration: 5 seconds
```

Total Chunks:
 $60 \times 60 / 5 = 720$ chunks

Each chunk retains its position in the original recording through timestamps.

Example:

Chunk 1 → 00:00 - 00:05
Chunk 2 → 00:05 - 00:10
Chunk 3 → 00:10 - 00:15
...
Chunk 720 → 59:55 - 60:00

This allows detected events to be mapped back to the exact position in the original proctoring recording.

9. Audio Detection Agent

The Audio Detection Agent will be implemented using **LangGraph**.

LangGraph will manage the agent's state, nodes, transitions, and decision-making workflow.

Core Agent Pipeline

```
Retrieve
↓
Analyze
↓
Reason
↓
Label
↓
Store
↓
Update Context
↓
Next Chunk / End
```

10. Agent State

The LangGraph state may contain the following information:

```
current_chunk
chunk_id
audio_file_id
source_file
chunk_index
start_timestamp
end_timestamp
analysis_result
detected_event
assigned_label
confidence_score
processing_status
previous_labels
recent_events
processing_history
```

Conceptually:

```
AgentState = {
  "audio_file_id": "...",
  "chunk_id": "...",
  "current_chunk": "...",
  "start_timestamp": 0.0,
  "end_timestamp": 5.0,
  "analysis_result": "...",
  "detected_event": "...",
  "assigned_label": "...",
  "confidence_score": 0.0,
  "processing_status": "...",
  "recent_events": [],
  "processing_history": []
}
```

11. LangGraph Nodes

The workflow can be divided into the following nodes.

11.1 Retrieve Chunk Node

Responsibilities:

- Request the next or required chunk from the Audio RAG Tool.
- Load the chunk and its metadata.
- Update the agent state.

INPUT:
Current processing state

OUTPUT:
Audio chunk + metadata

11.2 Analyze Audio Node

Responsibilities:

- Pass the retrieved audio chunk to the audio analysis component.
- Extract the predicted sound event.
- Obtain a confidence score where supported.

INPUT:
Audio chunk

OUTPUT:
Detected audio event
Confidence score
Analysis information

11.3 Reason Node

The Reason Node evaluates the analysis result and determines the next action.

Possible decisions include:

- Detection is sufficiently confident → assign label.
- Detection is uncertain → perform further analysis.
- Detection requires human review → mark for review.
- Chunk processing is complete → store result.
- More chunks remain → retrieve the next chunk.
- No chunks remain → end workflow.

11.4 Label Node

The Label Node maps the detected event to a standardized system label.

Example:

Detected: Typing sound
Label: KEYBOARD

Detected: Person speaking
Label: HUMAN_SPEECH

Detected: Traffic noise
Label: VEHICLE_NOISE

Detected: No significant sound
Label: SILENCE

11.5 Store Result Node

The Store Result Node saves the structured analysis result.

The stored result should include:

- Audio file ID
- Chunk ID
- Start timestamp
- End timestamp
- Detected event
- Assigned label
- Analysis result
- Confidence score
- Processing status

11.6 Update Context Node

The agent updates information about previously processed chunks.

Possible context includes:

- Previous labels
- Recent events
- Consecutive speech events
- Processing history
- Review flags

This helps identify continuous events across chunk boundaries.

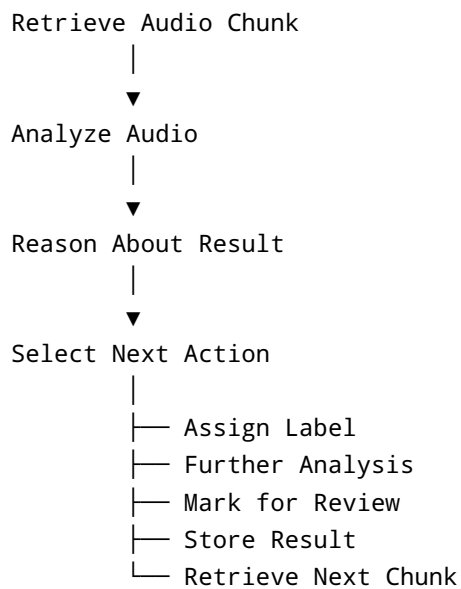
Example:

Chunk 10 → HUMAN_SPEECH
Chunk 11 → HUMAN_SPEECH
Chunk 12 → HUMAN_SPEECH

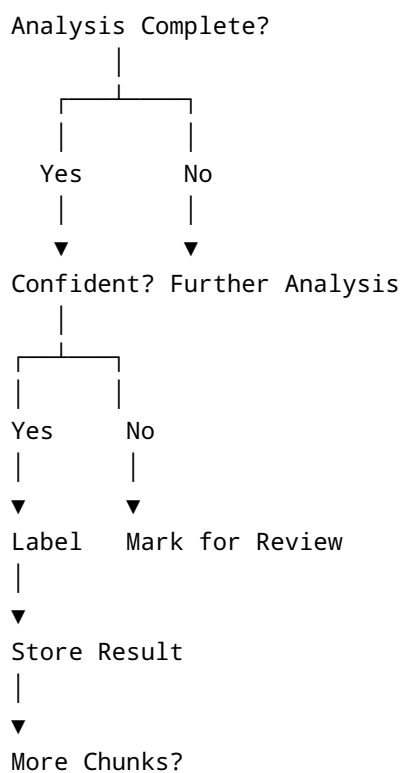
Instead of treating these as three completely independent events, the system can identify them as one continuous speech period.

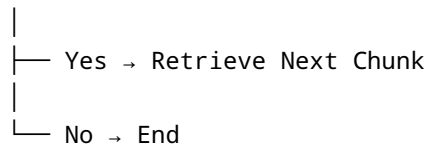
12. Reason-Action Loop

The agent follows a Reason-Action pattern during processing.

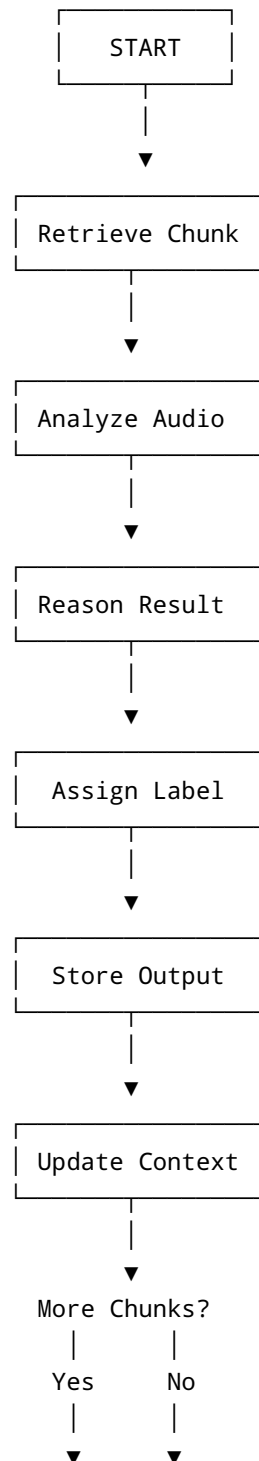


Decision Logic





13. Complete Agent Workflow



Retrieve Next END
Chunk

14. Expected Output

Each processed chunk will generate a structured output.

Example:

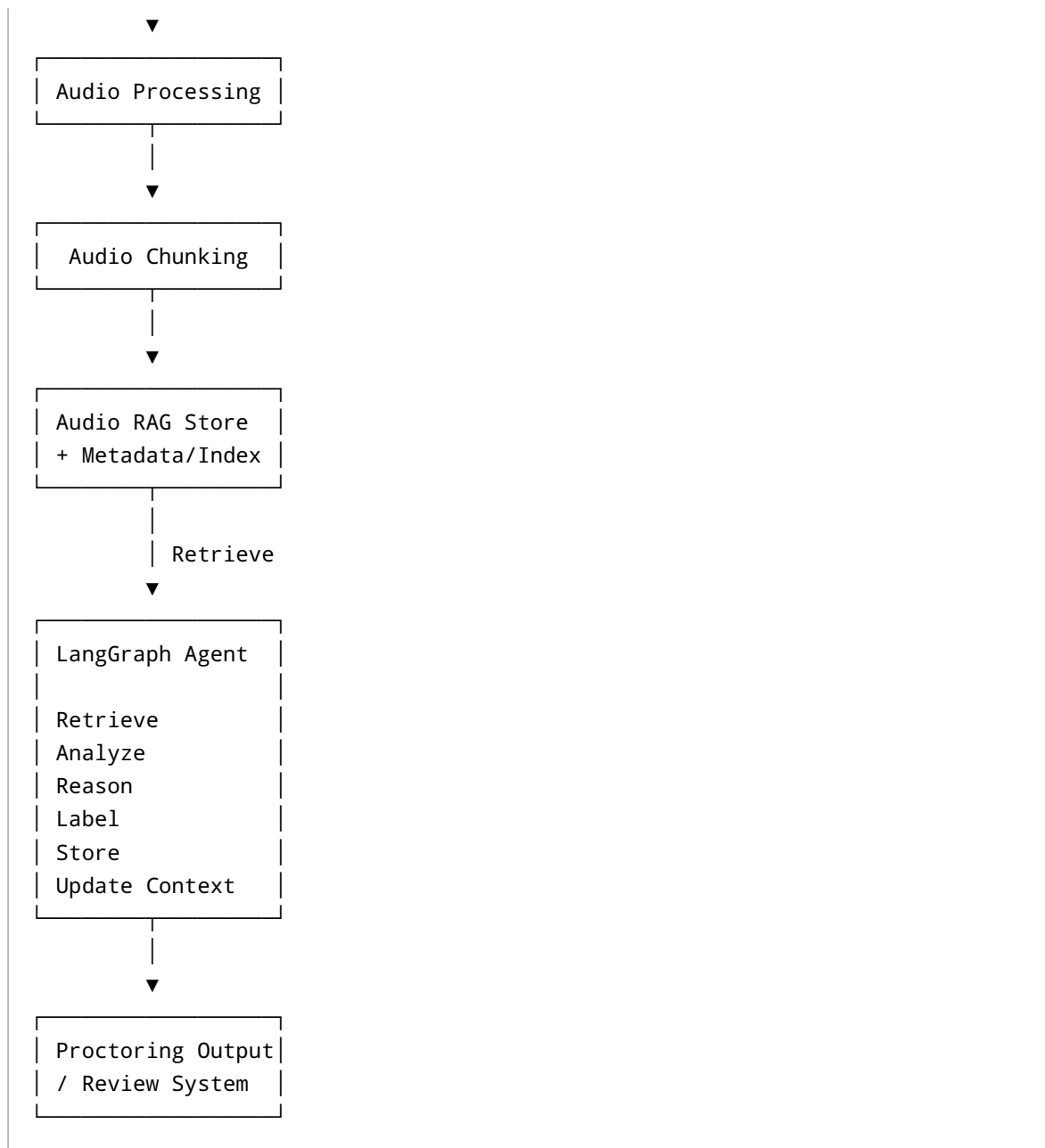
```
{
  "audio_file_id": "audio_001",
  "chunk_id": "chunk_00125",
  "chunk_index": 125,
  "start_timestamp": 600.0,
  "end_timestamp": 605.0,
  "detected_event": "Person speaking",
  "assigned_label": "HUMAN_SPEECH",
  "analysis_result": "Human speech detected in the audio segment.",
  "confidence_score": 0.94,
  "processing_status": "COMPLETED",
  "review_required": false
}
```

For uncertain detections:

```
{
  "audio_file_id": "audio_001",
  "chunk_id": "chunk_00126",
  "start_timestamp": 605.0,
  "end_timestamp": 610.0,
  "detected_event": "Uncertain environmental sound",
  "assigned_label": "OTHER",
  "confidence_score": 0.42,
  "processing_status": "REVIEW_REQUIRED",
  "review_required": true
}
```

15. High-Level Data Flow

Audio Upload



16. Implementation Scope

Phase 1: Audio Processing

- Load audio files.
- Validate supported audio formats.
- Preprocess audio if required.
- Split recordings into chunks.
- Generate timestamps and metadata.

Phase 2: Audio Storage and RAG

- Store chunk files.

- Store chunk metadata.
- Create an indexing/retrieval mechanism.
- Implement chunk retrieval APIs or tools.

Phase 3: Audio Analysis

- Integrate an audio classification or sound-event detection model.
- Detect supported audio events.
- Generate confidence scores.
- Standardize output labels.

Phase 4: LangGraph Agent

- Define agent state.
- Implement retrieval node.
- Implement analysis node.
- Implement reasoning node.
- Implement label assignment node.
- Implement result storage node.
- Implement context update node.
- Define conditional transitions.

Phase 5: Integration

- Connect Audio RAG with the LangGraph agent.
- Process complete recordings sequentially.
- Store all chunk-level results.
- Generate proctoring-ready output.

17. Future Enhancements

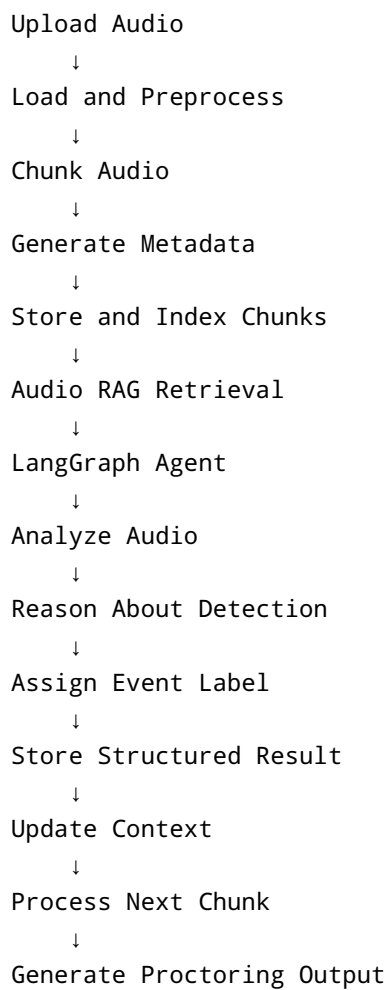
Possible future improvements include:

- Overlapping audio chunks to reduce boundary detection issues.
- Multiple simultaneous sound labels.
- Speaker count estimation.
- Voice activity detection.
- Continuous event merging across chunks.
- Priority or severity scoring for suspicious events.
- Real-time audio processing.
- Human review dashboard integration.
- Alert generation for suspicious patterns.
- Expanded sound-event label categories.
- Improved retrieval using audio embeddings.

18. Final Outcome

The final system will provide an **Audio RAG-based detection pipeline** integrated with a **LangGraph Audio Detection Agent**.

The complete processing flow will be:



The resulting system will enable efficient processing of long proctoring recordings and provide timestamped, structured audio-event detections for automated monitoring and human review.